

Esercizio A1

Limite asintotico inferiore Data la componente non ricorsiva, è facile vedere che $T(n) = \Omega(n^2)$.

Limite asintotico superiore Supponiamo che $T(n) = O(n^2)$. Dobbiamo dimostrare che

$$\exists c > 0, \exists m > 0 : T(n) \leq cn^2, \forall n \geq m$$

Nei casi base:

$$T(1) = 1 \leq c \cdot 1^2 \Rightarrow c \geq 1$$

$$T(2) = 2 \leq c \cdot 2^2 \Rightarrow c \geq 1/4$$

$$T(3) = 3 \leq c \cdot 3^2 \Rightarrow c \geq 1/9$$

Ipotesi induttiva:

$$T(k) \leq ck^2, \forall k < n$$

Passo induttivo:

$$\begin{aligned} T(n) &= T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 2T\left(\left\lfloor \frac{n}{2\sqrt{2}} \right\rfloor\right) + 7T\left(\left\lfloor \frac{n}{4} \right\rfloor\right) + n^2 \\ &\leq cn^2/4 + 2cn^2/8 + 7cn^2/16 + n^2 \\ &= \frac{15}{16}cn^2 + n^2 \\ &\leq cn^2 \end{aligned}$$

L'ultima disequazione è vera per $c \geq 16$. Abbiamo quindi dimostrato che

$$\exists c > 0, \exists m > 0 : T(n) \leq cn^2, \forall n \geq m$$

per $c \geq 16, m = 1$.

Conclusioni Abbiamo quindi dimostrato che $T(n) = \Theta(n^2)$.

Esercizio A2

Se l'albero binario in input è completo, è sufficiente controllare che il valore di uno nodo sia superiore a quello del padre. La procedura ricorsiva `mhRec()` prende in input il nodo sotto controllo e il valore del padre, verifica che la condizione sia rispettata nel nodo e verifica ricorsivamente nei due figli, passando il proprio valore. Il caso base è dato da un nodo **nil**, che ritorna **true**. Potendo assumere che l'albero sia non vuoto, la funzione wrapper richiama `mhRec()` sui due figli della radice.

```
boolean isMinHeap(TREE T)
```

```
    return mhRec(T.left, T.value) and mhRec(T.right, T.value)
```

```
boolean mhRec(TREE t, int lb)
```

```
    if T == nil then
```

```
        | return true
```

```
    else
```

```
        | return t.value > lb and mhRec(t.left, t.value) and mhRec(t.right, t.value)
```

La complessità è quella di una visita in profondità dell'albero, ovvero $\Theta(n)$.

Esercizio A3

Innanzitutto, è necessario calcolare una stima superiore alla dimensione del vettore. Questo si può fare controllando la posizione a distanze esponenzialmente crescenti. Non appena si trova un valore $\perp$, si usa l'indice così trovato come stima superiore della dimensione.

Utilizziamo poi un algoritmo di ricerca binaria per individuare l'indice del primo valore positivo o nullo nel vettore.

```
int countNegatives(MISTERYARRAY A)
```

```
    int n = 1
    while A[n]  $\neq \perp$  do
        | n = n · 2
    return cnRec(A, 1, n) - 1
```

```
cnRec(MISTERYARRAY A, int i, int j)
```

```
    if i == j then
        | return i
    int m =  $\lfloor (i + j) / 2 \rfloor$ 
    if A[m] < 0 then
        | return cnRec(A, m + 1, j)
    return cnRec(A, i, m)
```

Il ciclo while contenuto nel wrapper esegue al più $O(\log n)$ passi, in quanto la dimensione n così trovata è tale che $n < 2n$.

La procedura di ricerca ha costo $O(\log n)$, essendo una ricerca dicotomica.

Esercizio B1

È sufficiente costruire un vettore che contenga il massimo dei due elementi di ogni colonna e applicare la soluzione di Hateville al vettore così costruito. Infatti, se una casella viene selezionata su una colonna, non sarà possibile prendere l'altra casella della colonna stessa e nessun valore delle colonne precedente e successiva; ma in questo caso, tanto varrà prendere il massimo fra i due valori per ogni colonna e chiamare Hateville. La complessità dell'algoritmo proposto è $O(n)$.

```
int maxGain(int[][] A, int n)
```

```
    int[] D = new int[1...n]
    for i = 1 to n do
        | D[i] = max(A[1][i], A[2][i])
    return hateville(D, n)
```

Esercizio B2

Per raggiungere il lato in basso a destra, dobbiamo fare n passi a destra e n passi in basso, intervallati in qualunque modo.

Sia $DP[i][j]$ il numero di percorsi in una griglia di dimensione $i \times j$, potendo fare i passi lungo un asse e j passi lungo l'altro. È possibile esprimere ricorsivamente DP tramite la formula seguente:

$$DP[i][j] = \begin{cases} 1 & i = 0 \text{ or } j = 0 \\ DP[i-1][j] + DP[i][j-1] & \text{altrimenti} \end{cases}$$